

Projektuebersicht Superette

ERP-Supermarkt – Projektübersicht

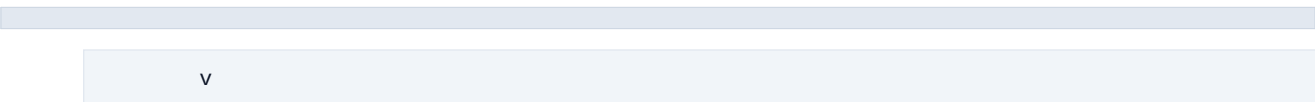
1. Projekt in einem Satz

Entwicklung einer vollständigen Datenlösung für eine kleine Superette – von der Analyse der Geschäftsprozesse über das relationale Datenmodell und die PostgreSQL-Datenbank bis hin zur ERP-Anwendung mit Streamlit und interaktiven Power BI-Dashboards.

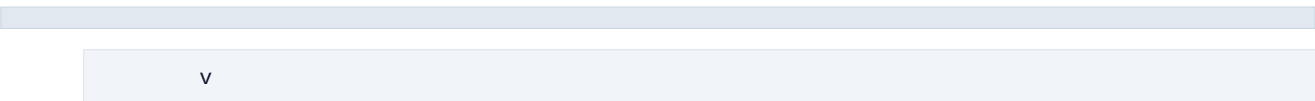
2. Projektarchitektur

Geschäftsanforderungen

(Ladenbesitzer)

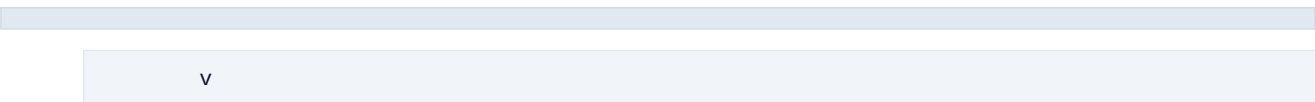


Anforderungsanalyse



ERD / Datenmodell

(selbst entwickelt)



PostgreSQL

(Datenbankdesign)



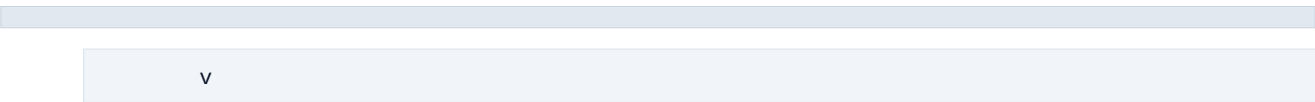
ETL Streamlit ERP

(Erstimport) (Produktivbetrieb)



database

(CRUD + SQLAlchemy)



utils

(Geschäftslogik)

v

Power BI

(Dashboards & Reports)

v

Neue Anforderungen

+-----■ zurück zum ERD

3. Ordnerübersicht

config/

Konfiguration der gesamten Anwendung.

- settings.py → Projekteinstellungen
- database.py → Datenbankverbindung
- auth.py → Benutzeranmeldung
- styles.py → Gemeinsames Layout

data/

Enthält alle Quelldaten.

- csv → Importdaten
- excel → Originaldateien
- backup → Sicherungen

database/

Data Access Layer (CRUD + SQL).

Datei Aufgabe

database_utils.py Gemeinsame SQL-Hilfsfunktionen

import_csv.py ETL-Import aller CSV-Dateien

run_import.py Startet den ETL-Prozess

reset_sequences.py Synchronisiert PostgreSQL-Sequenzen

achats_db.py Einkäufe

acheteurs_db.py Käufer

vendeurs_db.py Verkäufer

categories_db.py Kategorien

produits_db.py Produkte

ventes_db.py Verkäufe

lignes_achat_db.py Einkaufspositionen

lignes_vente_db.py Verkaufspositionen

depenses_db.py Ausgaben

pertes_db.py Verluste

stock_db.py Lager

inventaire_db.py Inventur

tresorerie_db.py Kassenbewegungen

dashboard_db.py Dashboard-KPIs

analytics_db.py Analysen

rapports_db.py Berichte & Exporte

dates_db.py Datumstabelle

utils/

Geschäftslogik und Hilfsfunktionen.

- validation.py → Datenvalidierung
- calculs.py → Berechnungen
- dashboard.py → Dashboardlogik
- charts.py → Diagramme
- exports.py → Exporte
- imports.py → Import-Hilfsfunktionen
- produits.py → Produktlogik
- categories.py → Kategorienlogik
- achats.py → Einkaufslogik
- ventes.py → Verkaufslogik
- stock.py → Lagerlogik
- inventaire.py → Inventurlogik
- depenses.py → Ausgabenlogik
- pertes.py → Verlustlogik
- tresorerie.py → Kassenlogik
- helpers.py → Allgemeine Hilfsfunktionen

streamlit/

Enthält die komplette Streamlit-Anwendung.

- README.md → Dokumentation der Anwendung
- pages → Alle Seiten der Benutzeroberfläche

Jede Datei im Ordner pages entspricht einer Seite der Streamlit-App.

01 Accueil → Startseite

02 Produits → Produkte

03 Categories → Kategorien

04 Achats → Einkäufe

05 Lignes Achat → Einkaufspositionen

06 Ventes → Verkäufe

07 Lignes Vente → Verkaufspositionen

08 Depenses → Ausgaben

09 Pertes → Verluste

10 Tresorerie → Kasse

11 Inventaire → Inventur

12 Rapports → Berichte

13 Dashboard → KPIs und Kennzahlen

14 Administration → Verwaltung

15 A Propos → Projektinformationen

sql/

SQL-Skripte für die PostgreSQL-Datenbank.

- 01_Create_Database.sql → Erstellt die Datenbank
- 02_Import_CSV.sql → Importiert CSV-Daten
- 03_SQL_Analytics.sql → Analyseabfragen
- 04_Views.sql → SQL-Views

- 05_Functions.sql → SQL-Funktionen
- 06_Triggers.sql → Trigger
- 07_Indexes.sql → Indizes
- 08_Migration_Vente_Declassee.sql → Datenmigration
- README.md → SQL-Dokumentation

powerbi/

Power BI Dashboard und Berichte.

- Superette_ERP_Dashboard_v1.pbix → Dashboard
- theme.json → Eigenes Design
- images → Dashboard-Screenshots
- README.md → Dokumentation

reports/

Automatisch erzeugte Berichte.

- csv → CSV-Exporte
- excel → Excel-Berichte
- pdf → PDF-Berichte
- images → Exportierte Grafiken

tests/

Automatisierte Tests der Anwendung.

- test_database.py → Datenbanktests
- test_dashboard.py → Dashboardtests
- test_achats.py → Einkaufstests
- test_ventes.py → Verkaufstests
- test_stock.py → Lagertests
- test_utils.py → Tests der Hilfsfunktionen

logs/

Logdateien der Anwendung.

- database.log → Protokolliert Datenbankereignisse
- import.log → Protokolliert den ETL-Import

docs/

Zusätzliche Projektdokumentation.

- screenshots → Screenshots der Streamlit-Anwendung
- pdf → PDF-Versionen der Screenshots

images/

Bilder und Grafiken des Projekts.

- logo.png → Projektlogo
- readme → Bilder und Screenshots für die README-Dateien
- README.md → Dokumentation des Bildordners

4. Root-Dateien

app.py

Startet die Streamlit-Anwendung und verbindet alle Projektmodule.

README.md

Zentrale Projektdokumentation mit Projektübersicht, Architektur, Installation und Funktionsbeschreibung.

CHANGELOG.md

Dokumentiert die wichtigsten Versionen und Weiterentwicklungen des Projekts.

CONTRIBUTING.md

Beschreibt Regeln und Empfehlungen für die Mitarbeit sowie den Coding Style des Projekts.

LICENSE.txt

Enthält die Lizenzinformationen des Projekts.

requirements.txt

Enthält alle benötigten Python-Bibliotheken für den Betrieb der Anwendung.

requirements-dev.txt

Zusätzliche Python-Bibliotheken für Entwicklung und Tests.

.env

Persönliche Konfigurationsdatei mit Datenbankzugang und Umgebungsvariablen.

.env.example

Beispieldatei für die Konfiguration ohne vertrauliche Daten.

.gitignore

Legt fest, welche Dateien und Ordner nicht in GitHub gespeichert werden.

5. Zusammenfassung

Wenn ich mein Projekt in wenigen Minuten vorstellen müsste, würde ich es folgendermaßen erklären:

"Ich habe für eine kleine Superette ein vollständiges ERP-System entwickelt, das heute im täglichen Betrieb eingesetzt wird. Ziel war es, die Geschäftsprozesse wie Produktverwaltung, Einkäufe, Verkäufe, Lagerverwaltung, Inventur, Betriebsausgaben und Kassenführung digital abzubilden.

Zu Beginn habe ich die Anforderungen gemeinsam mit dem Ladenbesitzer analysiert. Auf dieser Grundlage habe ich das Entity-Relationship-Diagramm (ERD) selbst entworfen und daraus das relationale Datenmodell entwickelt.

Anschließend habe ich die komplette PostgreSQL-Datenbank aufgebaut – einschließlich Tabellen, Primär- und Fremdschlüsseln, Beziehungen, Views, Funktionen, Triggern und Indizes.

Für die Übernahme der bereits vorhandenen Geschäftsdaten habe ich einen

ETL-Prozess entwickelt, der die Excel- und CSV-Dateien in die neue Datenbank importiert hat.

Nach dieser einmaligen Datenübernahme erfolgt die tägliche Arbeit direkt über die Streamlit-Anwendung. Neue Produkte, Einkäufe, Verkäufe, Inventuren und Betriebsausgaben werden dort erfasst und unmittelbar in der PostgreSQL-Datenbank gespeichert.

Die Anwendung ist modular aufgebaut. Der Datenzugriff erfolgt über den Ordner database, während die Geschäftslogik im Ordner utils umgesetzt ist. Dadurch lässt sich das System einfach warten und erweitern.

Für Auswertungen und Managementberichte nutze ich Power BI. Dort werden Kennzahlen wie Umsätze, Lagerbestände, Einkäufe, Ausgaben und weitere Geschäftsdaten interaktiv visualisiert.

Da die Anwendung im Einsatz ist, wird sie kontinuierlich weiterentwickelt. Neue Anforderungen des Ladenbesitzers fließen in das Datenmodell, die Datenbank und die Streamlit-Anwendung ein.

Mit diesem Projekt wollte ich zeigen, dass ich eine komplette Datenlösung entwickeln kann – von der Anforderungsanalyse über die Datenmodellierung und den Datenbankentwurf bis zur Entwicklung der Anwendung sowie der Analyse und Visualisierung der Daten."

6. Wichtige Technologien

Programmiersprache

- Python

Datenbank

- PostgreSQL

Datenbankzugriff

- SQLAlchemy
- psycopg2

Datenanalyse

- Pandas

Benutzeroberfläche

- Streamlit

Business Intelligence

- Power BI

SQL

- Tabellen
- Views
- Funktionen
- Trigger
- Indizes

Entwicklung

- Git
- GitHub
- Visual Studio Code

7. Wichtigste Kompetenzen

- ✓ Datenmodellierung
- ✓ Relationale Datenbanken
- ✓ SQL
- ✓ ETL-Prozesse
- ✓ Python
- ✓ PostgreSQL
- ✓ CRUD-Operationen
- ✓ Streamlit
- ✓ Power BI
- ✓ Dashboard-Entwicklung
- ✓ Datenvisualisierung
- ✓ Datenanalyse
- ✓ Softwarearchitektur
- ✓ Git & GitHub
- ✓ Technische Dokumentation

Ende der Projektübersicht